Ich hätte gerne eine App mit der ich alle meine Einkaufsrechnungen erfassen kann.
Jeder einzelne posten soll mit Menge, einzelpreis Beschreibung erfasst werden.
Dann soll jeder dieser Zeilen auch eine Kategorie zugewiesen werden.
Ebenso sollten hier Tank und Restaurantrechnungen mit erfasst werden, wobei hier die jeweils nur der Gesamtbetrag aufgenommen werden muss.
Es soll eine Art Haushaltsbuch werden, mit der ich sehen kann worfür ich wie viel Geld ausgebe.
Die Rechnungen können in unterschiedlichen Sprachen sein. Meist Deutsch oder Spanisch. Ich will hier meine Kosten in den unterschiedlichen Länden vergleichen können, das ist aber nebensächlich da man das einfach durch trennen in zwei tenants oder ähnlich lösen kann.

Die Anwendung soll selbst gehostet werden. Entweder als kleine PHP app oder in einem Docker container.

Es sollte eine Progressiv Web App sein, da ich sie vom Handy aus bedienen möchte.
Hier möchte ich die Möglichkeit haben einfach ein Foto von der Rechnung zu machen. Ich brauche aber die Möglichkeit PDFs hochzuladen.
Schließlich - das muss aber nicht am Handy funktionieren - brauche ich die Möglichkeit händisch EInträge zu korrigieren oder Vorzunehmen.

Meine Vorgehensweise wäre jetzt wie folgt:
1) Wir sollten ein gemeinsames Verständnis davon haben was ich suche
2) Du solltest überprüfen ob es solche Apps bereits gibt
3) Falls nicht gehen wir etwas meh ins technische Detail und definieren alles so genau, sodass Du mir einen guten Prompt für Claude Code oder Codex geben kannst.
4) Claude Code und/oder Codex bauen mir die App :-)

Show less

16:12

Das klingt nach einem sehr praktischen Projekt! Ich habe deine Anforderungen gut verstanden. Lass mich kurz zusammenfassen, was du dir vorstellst, damit wir ein gemeinsames Bild haben:

**Was du brauchst: Ein persönliches Haushaltsbuch als PWA**

*Kernfunktion:* Erfassung von Einkaufsrechnungen mit einzelnen Positionen (Menge, Einzelpreis, Beschreibung, Kategorie) sowie vereinfachte Erfassung für Tank- und Restaurantrechnungen (nur Gesamtbetrag + Kategorie).

*Foto/Upload:* Vom Handy ein Foto der Rechnung machen → automatische Erkennung der Positionen (via KI/OCR). PDF-Upload ebenfalls möglich.

*Mehrsprachigkeit:* Rechnungen auf Deutsch und Spanisch, mit der Möglichkeit Ausgaben nach Land/Region zu vergleichen (über Mandanten oder Tags).

*Bearbeitung:* Manuelles Erfassen und Korrigieren von Einträgen (primär am Desktop).

*Hosting:* Self-hosted, entweder als PHP-App oder Docker-Container.

*Auswertung:* Übersicht, wofür wie viel Geld ausgegeben wird (nach Kategorie, Zeitraum, evtl. Land).

---

Bevor wir tiefer ins Technische gehen – lass mich kurz recherchieren, ob es bereits fertige Self-hosted Lösungen gibt, die das abdecken.

Searched the web >

Gut, ich habe einen guten Überblick. Hier ist meine Einschätzung:

---

## Ergebnis der Recherche

Es gibt durchaus ähnliche Tools – keines trifft aber exakt deinen Use Case:

**Receipt Wrangler** kommt am nächsten: Self-hosted, AI-gestütztes Receipt-Management mit OCR/KI-Scan, Smart Categorization und Multi-User-Support. GitHub Es unterstützt Bilder und PDFs, auch mehrseitige. Receiptwrangler Allerdings fehlt die zentralen Punkte: **keine Einzelposten-Erfassung pro Zeile** (es erfasst den Gesamtbon, nicht einzelne Artikel mit Menge × Preis × Kategorie), kein Ländervergleich, und das Setup ist relativ komplex.

**BudgetLens** ist ein Open-Source Receipt-Scanner mit Django/Docker, der OpenAI für Bilderkennung nutzt und Rechnungen in beliebigen Sprachen verarbeitet. GitHub Aber: Cloud-abhängig (OpenAI/Exchange API), keine echte Einzelposten-Kategorisierung.

**Firefly III** ist ein vollständiger Self-hosted Finanzmanager mit Docker, Multi-Currency, Kategorien und REST API GitHub – aber ohne Foto-Scan und ohne Einzelposten-Logik.

**Fazit: Es gibt keine fertige Lösung**, die all deine Anforderungen abdeckt – insbesondere die **zeilenweise Erfassung mit Menge/Preis/Kategorie pro Artikel** + **Foto-Scan** + **Ländervergleich** + **PWA** in einem Package.

---

## Empfehlung: Eigene App bauen ✅

Das macht Sinn. Sollen wir jetzt zu **Schritt 3** übergehen und die technischen Details so präzise definieren, dass Claude Code einen guten Prompt bekommt?

Ein paar Fragen vorab, damit der Prompt wirklich präzise wird:

Q: Welche KI-API möchtest du für den Foto/PDF-Scan verwenden?
A: Mir stehen die OpenAI APIs als auch die Claude APIs zur Verfügung. Hier müsste man prüfen was aktuell hier am sinnvollsten ist.

Q: Wie soll die App technisch aufgebaut sein?
A: Was am sinnvollsten ist. Backend MariaDB oder SQLite. Warscheinlich SQLite.

Q: Welche Auswertungen sind dir wichtig? (Select all that apply)
A: Export als CSV/Excel, Ausgaben nach Kategorie (Kuchen/Balken)

Super, das reicht mir für einen präzisen Prompt. Kurz zur KI-API-Frage:

Searched the web >

Perfekt, ich habe jetzt alles was ich brauche. Hier meine KI-Empfehlung und dann der vollständige Prompt:

---

### KI-API Empfehlung: Claude Sonnet als primär, GPT-4o als Fallback

Claude hat bei OCR eine niedrigere Halluzinationsrate (0,09%), was für Rechnungserfassung wichtiger ist als rohe Geschwindigkeit – erfundene Preise wären fatal. Codesota Bei struktur-erhaltenden Aufgaben wie JSON-Ausgabe aus Dokumenten folgt Claude Anweisungen zuverlässiger als GPT-4o. Medium Außerdem hat Claude eine strikte Policy: Daten werden standardmäßig nicht für Training verwendet Koncile – gut für private Finanzdaten. Für Latein-Schriften (Deutsch/Spanisch) sind beide Modelle gleichwertig gut.

**Empfehlung:** Claude Sonnet als Standard, API-Key konfigurierbar (damit du auch GPT-4o nutzen kannst).

---

Jetzt der Prompt für Claude Code / Codex:

Claude Code Prompt – Haushaltsbuch PWA
Document

Der Prompt ist fertig! Hier eine kurze Zusammenfassung der wichtigsten Entscheidungen:

**Tech-Stack:** PHP 8.2 + SQLite + Vanilla JS – kein Framework, minimale Abhängigkeiten, läuft in einem einzigen Docker-Container. Nach `docker-compose up` sofort einsatzbereit.

**KI:** Claude Sonnet als Standard (niedrigere Halluzinationsrate = keine erfundenen Preise), GPT-4o per `.env` umschaltbar. Die KI gibt strukturiertes JSON zurück, das erst als editierbares Formular angezeigt wird – du musst also immer bestätigen bevor etwas gespeichert wird.

**Mandanten:** Du kannst "Deutschland" und "Spanien" als separate Tenants anlegen und jederzeit zwischen ihnen wechseln.

**Nächste Schritte:**

1. Den Prompt-Inhalt aus dem Artifact kopieren
2. Claude Code starten: `claude` im Terminal, dann den Prompt einfügen
3. Alternativ bei Codex (OpenAI): Prompt direkt einreichen

Willst du noch etwas am Prompt anpassen, bevor du loslegst? Zum Beispiel: Authentifizierung/Login hinzufügen, andere Standardkategorien, oder spezifische Auswertungen?

Wenn es in Docker läuft ist mir es egal ob es PHP ist oder nicht. Hier das sinnvollste verwenden.
Ist Die bewusst das GPT 4o schon lange obsolete ist. Vielleicht hier kurz mal eine Recherche machen welche MOdelle sinvoll sind und für den anwendungsfall möglichst kosten effizient. aber richtig, erfunde zahlen wären schlecht. man kann am schluss noch mal nachprüfen ob die summe übereinstimmt.
Currency wird immer EUR sein.
Das OCR und die Kategorisierung  müssen nicht real time sein. Reicht wenn die PWA das Foto erstmal erfasst oder die PDF und auf dem Server speichert. Dann kann später das Image verarbeitet werden und dann später wenn die ganze rechnung erfasst ist (oder mehrere) können die Kategorien zugeordnet werden, sofern es sinn macht die zwei dinge zu trennen.
Gggf. Gibt man die Kategorien vor, damit man nacher sinnvolle auswertungen machen kann.

Show less

Gute Punkte! Lass mich kurz die aktuellen Modelle recherchieren.

Searched the web >

Perfekt, jetzt habe ich ein klares Bild. Hier meine Analyse und dann der aktualisierte Prompt:

---

## KI-Modell Empfehlung (Stand März 2026)

Für unseren Use Case – asynchrone Rechnungsverarbeitung, kein Realtime-Zwang – ergibt sich folgende Empfehlung:

**Primär: Gemini 2.5 Flash** — Googles eigene Bezeichnung: "bestes Modell in Sachen Preis-Leistung" Google, mit nativer Multimodalität (kein separater OCR-Schritt nötig). Preis: $0.30 / 1M Input-Token, $2.50 / 1M Output-Token Price Per Token — etwa **5–10x günstiger** als Claude Sonnet oder GPT-4o. Bei gedrucktem Text (Kassenbelege!) führt Gemini zusammen mit Claude bei den Benchmark-Scores. AIMultiple

**Fallback: Claude Haiku 4.5** — Haiku 4.5 ist auf schnellen, kosteneffizienten Durchsatz ausgelegt Skywork und hat Claude's niedrige Halluzinationsrate. Teurer als Gemini Flash, aber günstiger als Sonnet.

**Bonus-Idee:** Da die Verarbeitung ohnehin asynchron läuft, kann man den **Batch-API-Rabatt** von Gemini nutzen: 50% Rabatt auf alle Paid-Modelle für asynchrone Verarbeitung Costgoat — macht Gemini 2.5 Flash noch günstiger.

**GPT-4o ist tatsächlich veraltet** – die aktuellen relevanten OpenAI-Modelle wären GPT-4o-mini oder GPT-5.1, aber bei diesem Use Case schlägt Gemini Flash sie preislich deutlich.

---

Jetzt der vollständig überarbeitete Prompt:

Claude Code Prompt – Haushaltsbuch PWA (v2)
Document · Version 2

Der Prompt ist aktualisiert. Hier die wichtigsten Änderungen zusammengefasst:

**Tech Stack:** PHP raus, Node.js 22 rein – deutlich moderner, besseres Ökosystem für async/Job-Queue-Logik, und in Docker ist der Unterschied eh egal.

**KI-Strategie:** Gemini 2.5 Flash als Standard ($0.30 / $2.50 pro Million Token Price Per Token – ein Bruchteil der Alternativen), Claude Haiku 4.5 als Fallback. GPT-4o komplett raus.

**Das Wichtigste – asynchrone Verarbeitung in 2 Phasen:**

- Phase 1: Foto/PDF sofort speichern → Nutzer bekommt direkt Rückmeldung
- Phase 2: Worker im Hintergrund kümmert sich um OCR → danach separat Kategorisierung
- Die Kategorisierung gibt der KI immer die **vorhandenen Kategorien vor**, damit die Auswertungen später kohärent sind
- Summen-Validierung am Ende: weicht KI-Ergebnis mehr als 2% vom Gesamtbetrag ab, wird der Nutzer informiert

**Währung:** EUR fix, komplett aus dem Schema entfernt.

Möchtest du jetzt mit Claude Code loslegen, oder gibt es noch Punkte die wir verfeinern sollten?

Bei den Kategorien wäre es mehr so wie
Gemüse, Obst, Butter, Limonaden, Säfte, Bier, Wein, Spiritosen, Gewürze, Brotwaren, Reinigungsmittel etc.

Das macht sehr viel Sinn – du willst echte **Produktkategorien** statt abstrakte Ausgaben-Kategorien. Dann kannst du später wirklich sehen "was gebe ich für Wein aus" oder "wie viel kostet mich Fleisch pro Monat".

Das ändert aber die Datenstruktur ein bisschen, weil wir jetzt zwei Ebenen brauchen:

**Ebene 1 – Hauptkategorie:** Lebensmittel, Drogerie, Haushalt, Tankstelle, Restaurant, etc. **Ebene 2 – Unterkategorie:** Gemüse, Obst, Butter, Bier, Wein, Reinigungsmittel, etc.

So kannst du sowohl "wie viel gebe ich insgesamt für Lebensmittel aus" als auch "wie viel davon für Wein" auswerten.

Außerdem – macht es für dich Sinn, die Kategorien **vorzudefinieren und fix zu halten** (damit die KI immer sauber zuordnet), oder willst du die Möglichkeit haben, jederzeit neue hinzuzufügen?

Q: Kategorie-Struktur: Eine oder zwei Ebenen?
A: Eine Ebene, aber feingranular (nur Wein, Obst...)

Q: Kategorien für nicht-Einkaufsrechnungen (Tank, Restaurant)?
A: Eigene Hauptkategorie (Tankstelle, Restaurant)

Q: Sollen die Kategorien editierbar/erweiterbar sein?
A: Ja, ich möchte selbst neue anlegen können

Perfekt, das ist die sauberste Lösung. Eine flache Liste, feingranular – die KI ordnet direkt "Wein" zu, und du kannst in der Auswertung selbst gruppieren wie du willst.

Ich aktualisiere jetzt den Prompt mit der finalen Kategorienliste und den angepassten Datenbankstrukturen:

Claude Code Prompt – Haushaltsbuch PWA
Document · Version 3

Gut, der Prompt ist aktualisiert. Die wichtigsten Designentscheidungen dahinter:

**Flach aber mit optionaler Gruppe:** Die `group_name`-Spalte ist nur ein Label für die Anzeige – kein eigener Fremdschlüssel, keine zweite Tabelle. Das hält die Datenstruktur einfach, aber du kannst in Charts trotzdem "alle Getränke zusammen" sehen wenn du willst.

**Tank & Restaurant als eigene Kategorien:** Sie sind gleichrangig mit "Wein" oder "Gemüse", tauchen in derselben Liste auf. Da Tank/Restaurant-Rechnungen ja keinen Einzelposten haben, wird die Kategorie direkt auf den Rechnungskopf gesetzt statt auf eine Position.

**KI-Kategorisierung ist zukunftssicher:** Weil der Prompt immer die aktuelle Kategorienliste aus der DB zieht, funktioniert die automatische Zuordnung auch wenn du später "Proteinpulver" oder "Hundefutter Premium" hinzufügst.

Der Prompt ist damit aus meiner Sicht vollständig und präzise genug für Claude Code. Willst du noch etwas anpassen, oder soll ich dir noch kurz erklären wie du Claude Code am besten damit startest?

Wir sollten sicherstellen, dass eine README oder AGENT.md vorhanden ist, die genau beschreibt was wir warum wollen.

Guter Punkt – eine `AGENT.md` ist bei Claude Code besonders wichtig, weil Claude sie automatisch liest und als dauerhaften Kontext nutzt. Das verhindert, dass bei längeren Sessions

oder Fortsetzungen Entscheidungen vergessen oder falsch getroffen werden.

Ich füge sie direkt als eigenes Dokument hinzu und ergänze den Prompt um die Anweisung, sie zu erstellen:

Claude Code Prompt – Haushaltsbuch PWA
Document · Version 4

AGENT.md – Haushaltsbuch PWA
Document

Jetzt haben wir zwei Dokumente: den **Prompt** für Claude Code und die **AGENT.md** die ins Projekt kommt.

Die AGENT.md erfüllt drei Funktionen gleichzeitig:

**Für Claude Code:** Wird automatisch als persistenter Kontext gelesen. Wenn du eine neue Session startest oder die App erweiterst, kennt Claude Code sofort alle Entscheidungen – warum SQLite, warum kein GPT-4o, warum asynchrone Verarbeitung, was die App bewusst *nicht* kann.

**Für dich:** Falls du in 6 Monaten etwas ändern willst, steht das "Warum" direkt dabei. Z.B. "Warum ist das flach und kein Baum?" → steht drin.

**Als Leitplanke:** Der Abschnitt "Was diese App NICHT ist" verhindert, dass Claude Code Feature-Creep betreibt und anfängt Login-Systeme oder Budgetlimits zu bauen.

**Nächster Schritt:** Den Inhalt des Prompt-Artifacts in Claude Code einfügen. Am besten so starten:

```
claude
```

Dann den kompletten Prompt-Text einfügen. Claude Code wird als erstes die AGENT.md anlegen und dann Schritt für Schritt die App bauen.

To run code, enable code execution and file creation in Settings > Capabilities.